

Data Structures

Greedy

Dynamic Programming

Data Structures

What's a data structure?

(Wikipedia) “a collection of **data values** , the **relationships** among them, and the functions or **operations** that can be applied to the data”

- an **abstraction** of data that is easy for a programmer to work with
- contains more than data: the data is **organized** in a specific way
- well-defined **operations** can be applied to the data
 - it is important to know what data structures exist and which operations can be applied on them

Data Structures “covered” in this course :

Linear Structures :

- Dynamic Array
- Linked List
- Stack
- Queue

Associative Structures :

- Hash Table / Dictionaries
- Set

Dynamic arrays (Python lists)

Goal: quickly **access indexed items** in a container and **append new ones** (or remove the last one)

Implementation:

- the language uses more room than needed
- while there is room, appending costs nothing
- when there is no more room, create a new array with more room and copy everything



Dynamic arrays (Python lists)

What's the complexity of adding a new item?

if you reach the capacity c_1 and extend the size to $c_2 = 2 * c_1$, the cost is c_1 only once but then the cost will be const for the next c_1 *append* operations

→ on average, the cost is $O(1)$

What's the complexity of accessing an item?

$O(1)$, position in RAM deduced from its index



Dynamic arrays: in Python and C++

<i>Common operations:</i>	Python	Complexity
Access the i -th item	<code>arr[i]</code>	$O(1)$
Add v at the end	<code>arr.append(v)</code>	$O(1)$ avg $O(n)$ max
Insert v at position i	<code>arr.insert(i, v)</code>	$O(n)$
Find the position of v	<code>arr.index(v)</code>	$O(n)$

When to use:

There are all kinds of use cases. If you often need to perform operations that are not $O(1)$, check if another data structure matches your needs

Quick implementation cheat sheet - Python array

	Instruction	Complexity
Create a new one	<pre>L = list(), L = []</pre>	$O(1)$
Access the i-th item	<pre>arr[i]</pre>	$O(1)$
Add v at the end	<pre>arr.append(v)</pre>	$O(1)$ avg $O(n)$ max
Insert v at position i	<pre>arr.insert(i, v)</pre>	$O(n)$
Find the position of v	<pre>arr.index(v)</pre>	$O(n)$

Quick implementation cheat sheet - Python array

	Instruction	Complexity
Create a new one	<pre>L = list(), L = []</pre>	$O(1)$
Remove the first item with value x	<pre>arr.remove(x)</pre>	$O(n)$
Remove the item at the i-th position	<pre>arr.pop(i)</pre>	$O(n)$
Clear the array	<pre>arr.clear()</pre>	$O(n)$
Get the number of items	<pre>arr.len()</pre>	$O(1)$
Sort	<pre>arr.sort(key=function)</pre>	$O(n*\log(n))$
Count the number of items with value x	<pre>arr.count(x)</pre>	$O(n)$

Linked list

A linked list is a sequence of nodes that contains 2 fields :

- Some **data** (an integer value, here as an example)
- A **link** to the next node (the last node is linked to a terminator used to signify the end of the list)



Linked list

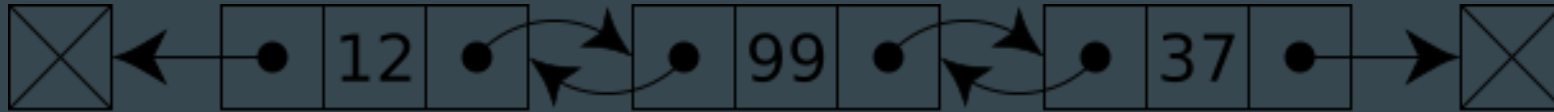
They can be used to implement several other common data types (stacks, queues, etc.)

The principal benefit of a linked list over a conventional array is that the list elements can be easily inserted or removed without reallocation or reorganization of the entire structure because the data items do not need to be stored contiguously

On the other hand, since simple linked lists by themselves do not allow random access to the data or any form of efficient indexing, many basic operations—such as obtaining the last node of the list, finding a node that contains a given datum, or locating the place where a new node should be inserted—may require iterating through most or all of the list elements

Linked list - Variant examples

Doubly linked list :



Circular linked list :



What is the common point between these pictures ?



What is the common point between these pictures ?



They're all **stacks** !!!

Stack

A stack has 2 main operations :

- **Push**, which adds an element on top
- **Pop**, which removes the most recently added element

A stack is a **LIFO** : **L**ast **I**n **F**irst **O**ut

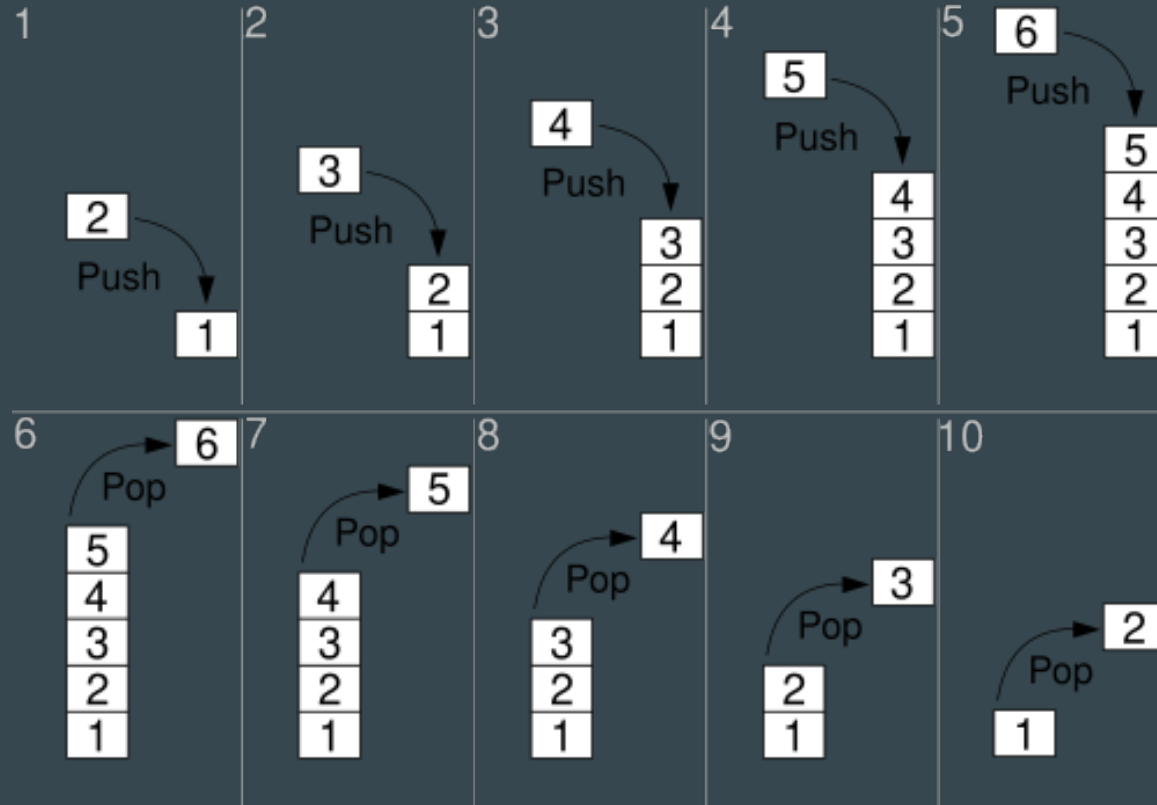
Just think of stackable objects (books, plates, boxes, etc.)

You can use python lists as stacks :

- `list.append()` = push
- `list.pop()` = pop

For better performance you can use the *collections.deque*

Stack - Visual exemple



Queues

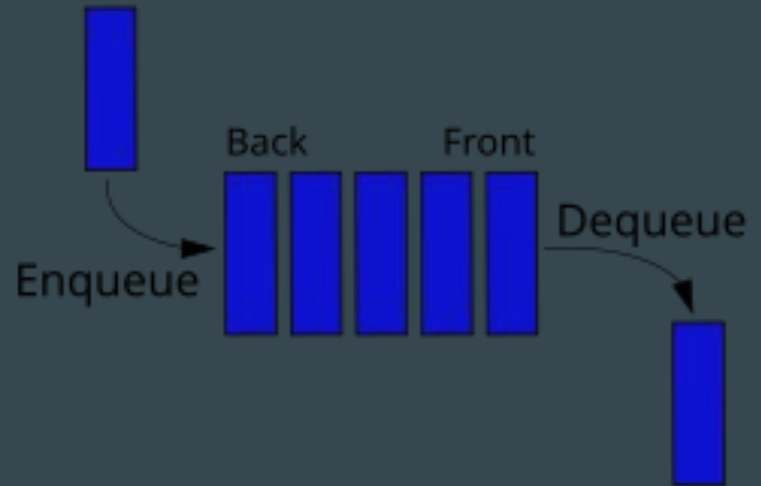
A queue has 2 main operations :

- **Enqueue** , which adds one element to the rear of the queue
- **Dequeue** , which removes one element from the front of the queue

A queue is a **FIFO** : **F**irst **I**n **F**irst **O**ut

Just think of a **waiting line** :

- Supermarket checkout
- Traffic light
- etc.



Queue - A python approach

You can use python lists as queues :

- `list.append()` = enqueue (**$O(1)$**)
- `list.pop(0)` = dequeue (**$O(n)$** !!)

For better performance you can use `collections.deque` :

- `deque.append()` = enqueue (**$O(1)$**)
- `deque.popleft()` = dequeue (**$O(1)$**)

Hash table / Dictionaries

Fist, don't mix up **hash table** and **dictionary**

Hash table = the concept, the data structure

Dictionary = a practical implementation in python

Hash table / Dictionaries

Goals:

- associate **values** to **keys**
- retrieve the value associated to a key in $O(1)$ time

→ unlike real-life dictionaries, these ones are **not ordered**

(in real life, finding a word in a dictionary is $O(\log n)$
unless you have forgotten the alphabetical order)

Hash table / Dictionaries

The underlying structure is called a **hash table**

- a hash function turns the keys into an index
- the keys and values are stored in the data structure based on this index
- multiple strategies exist to manage collisions (cf next slides)

Example of hash function: sum of the ASCII codes for a string, modulo 42

“hello” $\rightarrow$ 28

“world” $\rightarrow$ 6

“INSAIgo” $\rightarrow$ 33

Hash table / Dictionaries - separate chaining

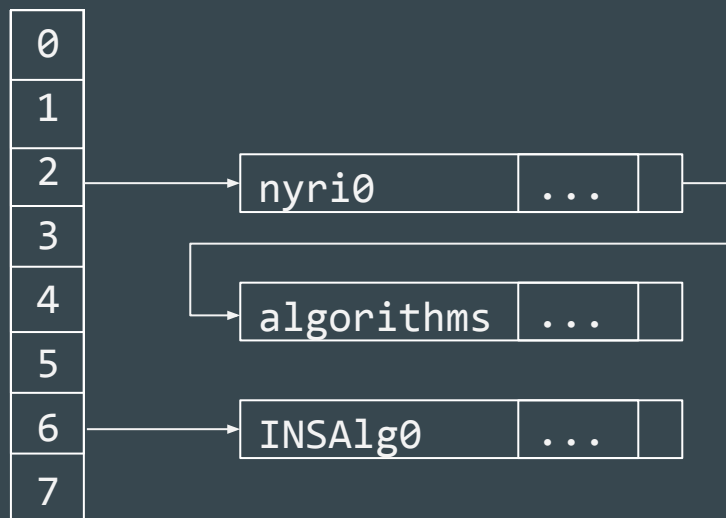
(key, value) couples where keys have the same hash are stored in a common data structure

These structures are kept small enough to have $O(1)$ time access. When too filled, the table is re-created bigger

Often used:

- linked lists
- trees

With sum of ASCII codes modulo 8, and using linked lists:



Hash table / Dictionaries - Open addressing

- No other data structure behind the array
- Collisions are solved with *probing*
- When the fill rate becomes too big, copy the data in a bigger hash table

Linear probing = put at next cell available

Randomized probing: follow a random sequence which seed is given by the hash

→ used in Python

0		
1		
2	2	nyri0 ...
3	2	algorithms ...
4		
5		
6	6	INSA1g0 ...
7		

(sum of ASCII codes modulo 8, open addressing with linear probing)

Hash table / Dictionaries

When to use:

Whenever you need to associate a value to a key.

In Python, the ease of use and high performance make the dictionaries a **very powerful tool** .

When not to use:

When your keys are 0, 1, ..., n
You're better than that.

Quick implementation cheat sheet - Python dict

Python: dict, built in.

Interesting variation : defaultdict in collections

	Python	Complexity
Create a dict	<code>d = dict(),</code> <code>d = {}</code>	O(1)
Access an element	<code>d[key]</code>	Pseudo O(1)
Put an element	<code>d[key] = value</code>	Pseudo O(1)
Find if a key is in dict	<code>key in d</code>	Pseudo O(1)
Remove a key	<code>del d[key]</code>	Pseudo O(1)

Sets

Goals:

- store unique values
- quickly check if a value is in the set or not

Two common implementations:

- tree-based sets are ordered
- hash sets are faster but unordered
(they're basically hash tables without values)

Quick implementation cheat sheet - Python set

Python: `set`, `built in` : implemented as a hash set

	Python	Complexity
Create a set	<code>S = set()</code>	$O(1)$
Access a specific element	IMPOSSIBLE	
Put an element	<code>s.add(element)</code>	Pseudo $O(1)$
Find if an element is in a set	<code>Element in s</code>	Pseudo $O(1)$
Remove an element	<code>s.remove(element)</code>	Pseudo $O(1)$

Set arithmetics in Python

		Average complexity
$s1 \leq s2, s1 < s2$	check if $s1$ is a [proper] subset of $s2$	$O(n1)$
$s1 \geq s2, s1 > s2$	check if $s1$ is a [proper] superset of $s2$	$O(n2)$
$s1 \mid s2 \mid \dots \mid sk$	union of $s1, s2, \dots, sk$	$O(n1 + n2 + \dots + sk)$
$s1 \& s2 \& \dots \& sk$	intersection of $s1, s2, \dots, sk$	$O(\min(n1, n2, \dots, sk))$
$s1 - s2$	all elements of $s1$ that are not in $s2$	$O(n1)$
$s1 \wedge s2$	all elements of $s1$ or $s2$ but not both	$O(n1 + n2)$

Sets too are cool :)

Sets - in Python

When to use:

- to mark values already seen
- to keep a collection of unordered unique elements

When not to use:

- to make coffee (you just can't)
- to keep twice the same element (really, you just can't)

Greedy

Greedy algorithm
...

cuz data tastes good



A simple coin change problem

You have 20\$, 10\$, 5\$, 2\$ and 1\$ bills

→ you want to pay N\$ with as few bills as possible

e.g $58\$ = 2 \cdot 20\$ + 1 \cdot 10\$ + 1 \cdot 5\$ + 1 \cdot 2\$ + 1 \cdot 1\$ \rightarrow 6$ bills

Strategy: let's try to use the biggest bills as long as possible



A simple coin change problem

Algorithm:

```
n = int(input())
bills = [20, 10, 5, 2, 1]
nbill = []

for bill in bills:
    nbill.append(n // bill)
    n -= nbill[-1] * bill

print(" + ".join("%d*%d$" % (nbill[i],
                                bills[i])
                  for i in
                  range(len(bills))))
```

Complexity:

$O(\text{len}(\text{bills})) \rightarrow \text{const}, O(1)$

Why it works:

The dollar system is called a *canonical* coin system

→ cf Xuan Cai,

<https://arxiv.org/pdf/0809.0400.pdf>

Non-canonical system: [9, 4, 1]

Optimal: $12 = 3*4$

Greedy: $12 = 1*9 + 3*1$

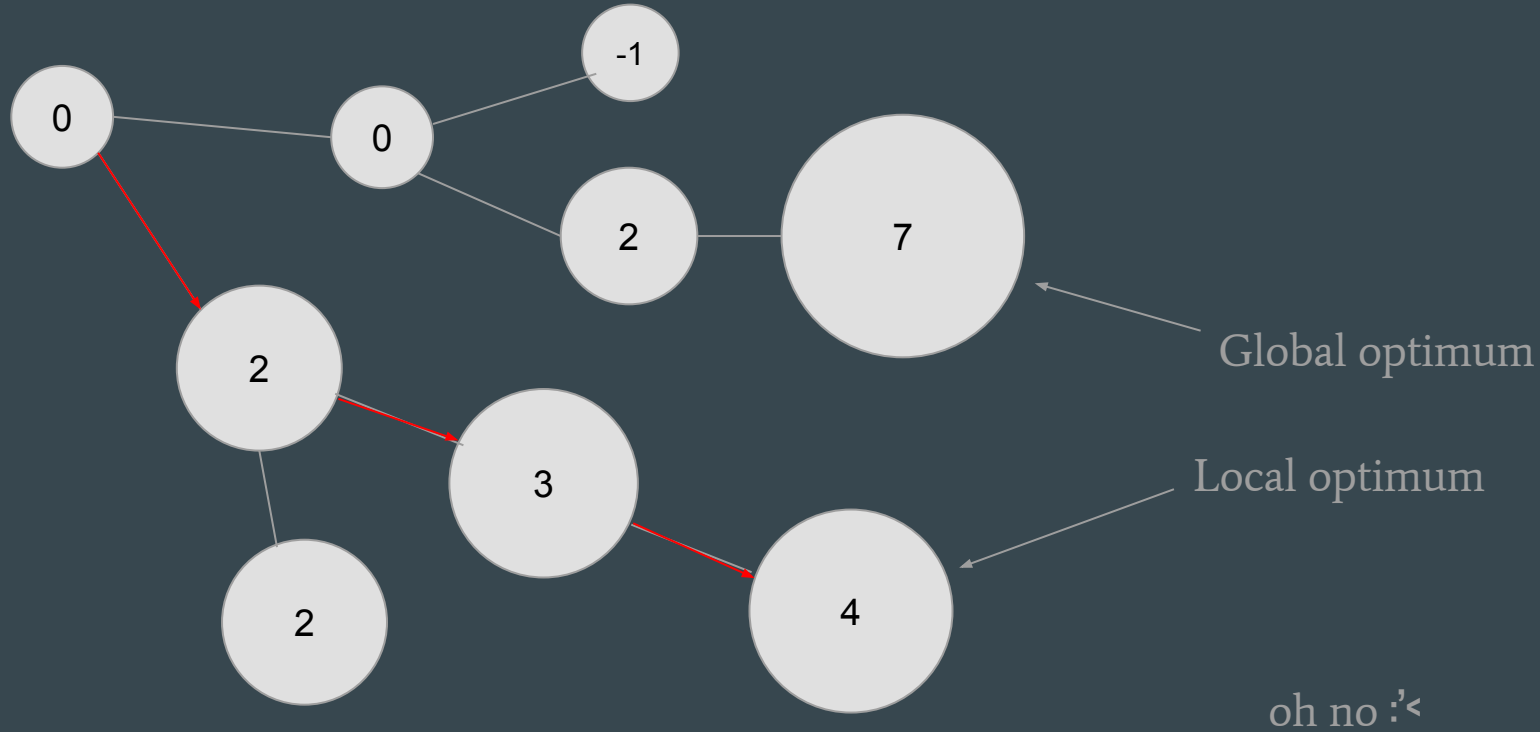


So what's a greedy algorithm?

- it's an algorithm that makes **locally-optimal choices** at each step
- the global solution might not be the optimal one
- but many well-known greedy algorithms are proven to find the optimal solution, e.g:
 - Kruskal's and Prim's algorithms (minimum spanning tree)
 - Dijkstra's and A* algorithms (shortest path)

Greedy algorithms are usually **much faster** than their *dynamic programming* equivalents, but their correctness can be hard to prove...

So what's a greedy algorithm? - Understand with the search space



Example of a non-optimal greedy algorithm

You want to climb on the highest mountain.

You think “if I keep going up, I will reach the highest point”.

You end up on the Fourvière hill.

Not quite as tall as Mount Everest.



Credits

Slides: Louis Sugy for INSAIgo, modified by Louis Hasenfratz



Pictures: Wikipedia

Picture of the cookie monster: fair use of the character from *Sesame Street*

Dynamic Programing

Dynamic Programming (DP)

Using recursive solution properties to trade compute time with memory

Definition

Dynamic Programming (DP) is a method for solving complex problems by breaking them into smaller, simpler subproblems.

- Each subproblem is solved once.
- The result is stored (memoized) to avoid recomputation.
- This makes algorithms more efficient by saving both time and effort.

Core Principles

Overlapping Subproblems

- The same smaller problems appear multiple times in computation.
- Example: Computing Fibonacci numbers requires repeatedly solving $F(n-1)$ and $F(n-2)$.

Optimal Substructure

- The optimal solution to the whole problem can be built from optimal solutions of its subproblems (like Greedy algorithms !!!)
- Example: Shortest path from A to C via B = shortest path from A to B + shortest path from B to C.

How to choose the subproblems ?

- The idea is to find how to recursively reach the easy problems, without losing optimality
- We want the result of the subproblems to be useful in order to compute the result of the greater one we're struggling with

Bellman's principle of optimality: if subproblems' result are optimal then their combination will be too

Bellman equations

Bellman equations are the rules to go from one subproblem to another.

- They need to go with Bellman's principle of optimality

Try smally decreasing any problem parameter (number of element, size of a line, ...)

The subproblems have to strictly decrease in size, leaning toward the base case

→ There cannot be any cycle !

- Don't forget the base case, The recursion (induction) has to stop at some point !

Alternative names (more explicit)

- **Memoization-Based Programming:** Highlights the reuse of intermediate results to avoid redundant computation.
- **Subproblem Optimization Programming:** Emphasizes breaking problems into smaller subproblems and solving them optimally.
- **Overlapping Subproblems Method:** Highlights the key feature of DP where subproblems overlap, making memoization or tabulation effective.
- **Iterative Refinement Programming:** Describes the process of incrementally building up solutions in the bottom-up approach.
- **Recursive Optimization Programming:** Combines the recursive nature and optimization focus of DP.

A common example: the Fibonacci sequence

Rules (Bellman equations) :

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2)$$

How to compute efficiently $F(n)$
for a large n ?

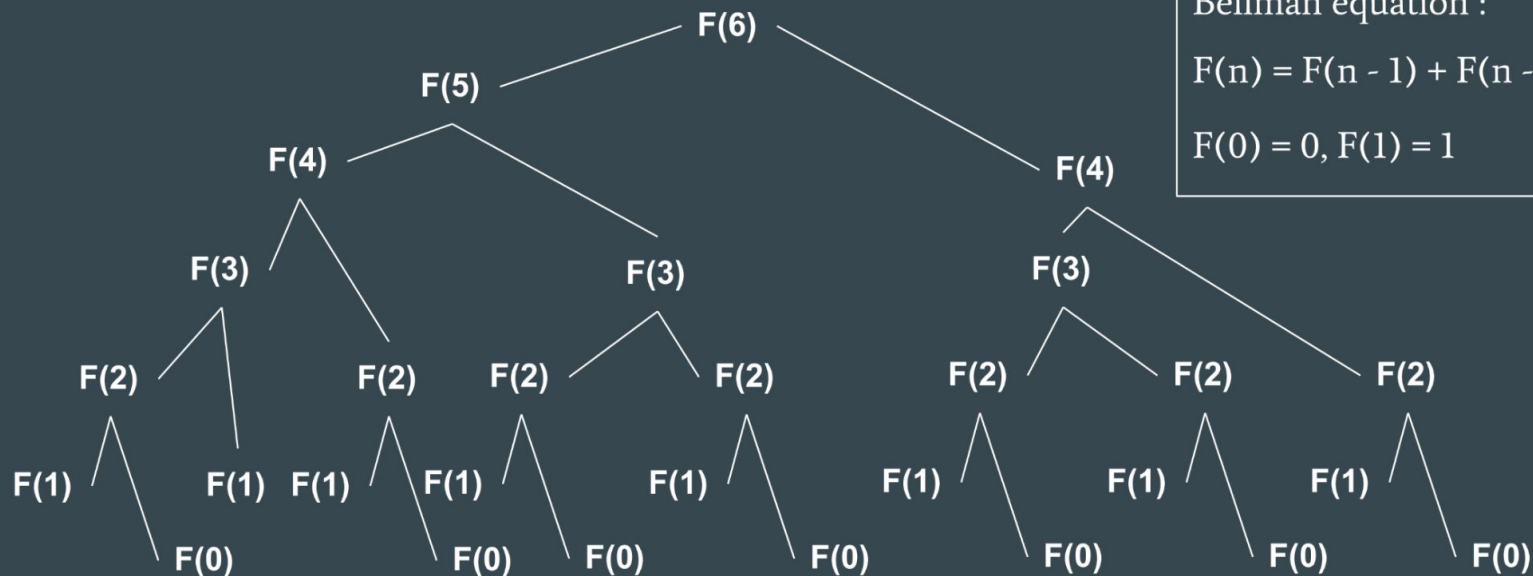


The Fibonacci sequence

Naive recursive algorithm :

```
def Fibonacci(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    else:  
        return Fibonacci(n - 1) + Fibonacci(n - 2)  
  
Fibonacci(n)
```

The Fibonacci sequence



Bellman equation :

$$F(n) = F(n - 1) + F(n - 2)$$

$$F(0) = 0, F(1) = 1$$

This algorithm runs in $O(2^n)$, there is probably a better way to do this..

Memoization: how to trade space for time

Keep track of which subproblems have already been solved

- Keep those subproblems' result in memory
- If one shows up again, take its result out of memory instead of computing it again

An efficient algorithm for computing the Fibonacci sequence

```
def Fibonacci(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    if memo[n] != -1:  
        return memo[n]  
    else:  
        result = Fibonacci(n - 1) + Fibonacci(n - 2)  
        memo[n] = result  
        return result  
  
memo = [-1] * (n + 1)  
Fibonacci(n)
```

An efficient algorithm for computing the Fibonacci sequence

```
def Fibonacci(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    if memo[n] != -1:  
        return memo[n]  
    else:  
        result = Fibonacci(n - 1) + Fibonacci(n - 2)  
        memo[n] = result  
        return result
```

```
memo = [-1] * (n + 1)  
Fibonacci(n)
```

<Base case>

<reuse already computed
subproblems if possible>

<Bellman equations>

<store result (memoize)>

Sequence of calls with this algorithm



Bellman equation :

$$F(n) = F(n - 1) + F(n - 2)$$

$$F(0) = 0, F(1) = 1$$

First call

Base case

Already computed

We've achieved a $O(n)$ algorithm !

Bottom Up vs Top Down approach

Bottom up

- Iterative version
- Build subproblems and grow toward the global one
- A bit faster, better for memory concerns



Initial problem

Base cases

Top down

- Recursive version
- Decrease from the initial problem to the base cases
- Easy to implement once you have the Bellman equation



Initial problem

Base cases

Bottom Up vs Top Down : Fibonacci sequence

Bottom Up

```
fibo = [0] * (n + 1)
fibo[0] = 0
fibo[1] = 1
for k in range(2, n + 1):
    fibo[k] = fibo[k - 1] + fibo[k - 2]
```

Top Down

```
def Fibonacci(n):
    if 0 <= n <= 1:
        return n
    if memo[n] != -1:
        return memo[n]
    else:
        result = Fibonacci(n - 1) + Fibonacci(n - 2)
        memo[n] = result
        return result

memo = [-1] * (n + 1)
Fibonacci(n)
```

What to remember from Dynamic Programming ?

DP is about trading space for time : we store results to avoid recomputation

Instead of solving problems from scratch each time, think about how smaller pieces can be reused

Widely applicable in algorithms, AI, optimisation problems, ...

Credits

Slides: Arthur Tondereau, Louis Sugy, Onyr (Florian Rascoussier) for INSAIgo

The algorithm design manual, Steven S.Skiena

Wikipedia of course!